

实验六：跨平台综合项目实战

- > **项目定位:** 智能健康运动记录平台 - 跨平台综合实践
- > **服务对象:** 运动爱好者、健康管理用户、开发学习者
- > **核心目标:** 掌握多技术栈开发、体验团队协作、学会AI辅助编码
- > **技术栈:** Android(Kotlin) + Uniapp(Vue) + 微信小程序
- > **项目周期:** 6学时
- > **团队规模:** 3人小组

目录

- 1. 项目背景与目标
- 2. 需求分析
- 3. 技术架构
- 4. 功能设计
- 5. 实施计划
- 6. 团队分工
- 7. 测试验证
- 8. 技术对比分析
- 9. 总结与体会
- 附录 A. 核心术语表
- 附录 B. 参考资源

1. 项目背景与目标

1.1 项目背景

随着移动互联网的发展，跨平台开发已成为移动应用开发的重要趋势。本实验旨在通过团队协作方式，让学生掌握多种主流跨平台技术栈的开发能力，体验真实的项目开发流程。

1.2 核心目标

```
+-----+
| FIG 1.2  PROJECT GOALS (ASCII only, PDF-safe) |
| Input: Team collaboration & AI-assisted development |
|           |           |           |           |
|           v           v           v           |
| [Multi-platform Skills] --> [Git Collaboration] |
|           |           |           |           |
|           +-----> [AI Coding] <---+           |
|           |           |           |           |
|           [Technical Comparison]           |
+-----+
```

1.3 学习价值

- **技术能力**：掌握3种主流移动开发技术栈
- **协作能力**：体验Git团队协作流程
- **工具能力**：学会使用AI辅助开发工具
- **分析能力**：完成技术选型对比分析

2. 需求分析

2.1 功能需求矩阵

功能模块	功能描述	技术要点
用户登录	模拟用户登录功能	状态管理、本地存储
步数统计	加加速度计数据采集与步数计算	传感器API、数据模拟
步频监测	实时步频数据展示	传感器API、UI展示
运动记录	运动历史记录列表	本地存储、列表渲染
数据同步	模拟数据同步机制	状态管理、数据持久化
AI建议	基于运动数据的个性化建议	API集成、对话交互
打卡功能	每日运动打卡记录	日期管理、状态持久化
社交动态	运动分享与互动	数据存储、用户交互

2.2 数据模型设计

```
// User.kt - 用户数据模型
data class User(
    val id: String,
    val name: String,
    val studentId: String? = null,
    val avatar: String? = null
)

// ExerciseRecord.kt - 运动记录数据模型
data class ExerciseRecord(
    val id: String,
    val userId: String? = null,
    val type: ExerciseType? = ExerciseType.RUNNING,
    val steps: Int,
    val distance: Double,
    val calories: Int,
    val cadence: Int = 0,
    val heartRate: Int = 0,
    val date: String,
    val duration: Int
)
```

```
enum class ExerciseType {  
    RUNNING  
}  
  
// StepData.kt - 步数数据模型  
data class StepData(  
    val steps: Int,  
    val distance: Double,  
    val calories: Int,  
    val cadence: Int = 0,  
    val timestamp: Long = System.currentTimeMillis()  
)
```

2.3 非功能需求

类别	要求
性能	应用启动时间 < 3秒
兼容性	各平台运行正常
用户体验	界面美观、操作流畅
数据安全	本地数据加密存储

3. 技术架构

3.1 整体架构

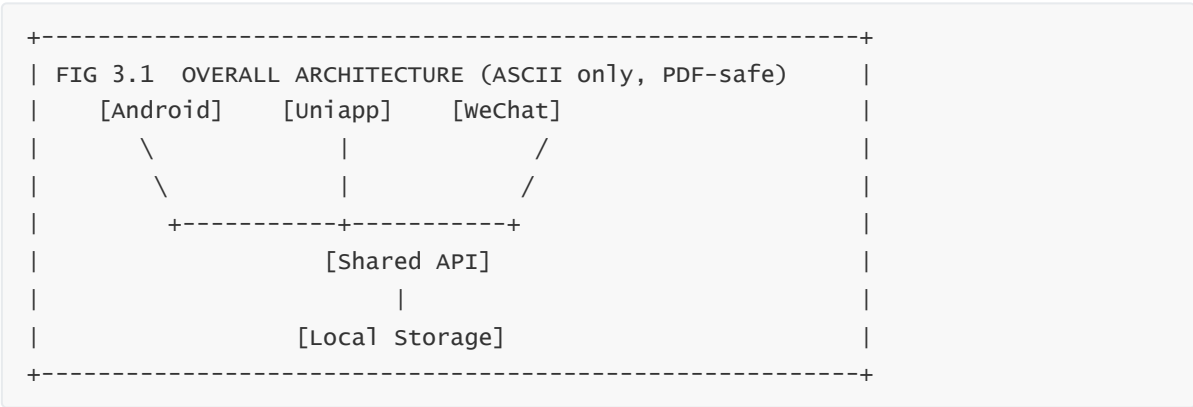


图3-1 系统架构图

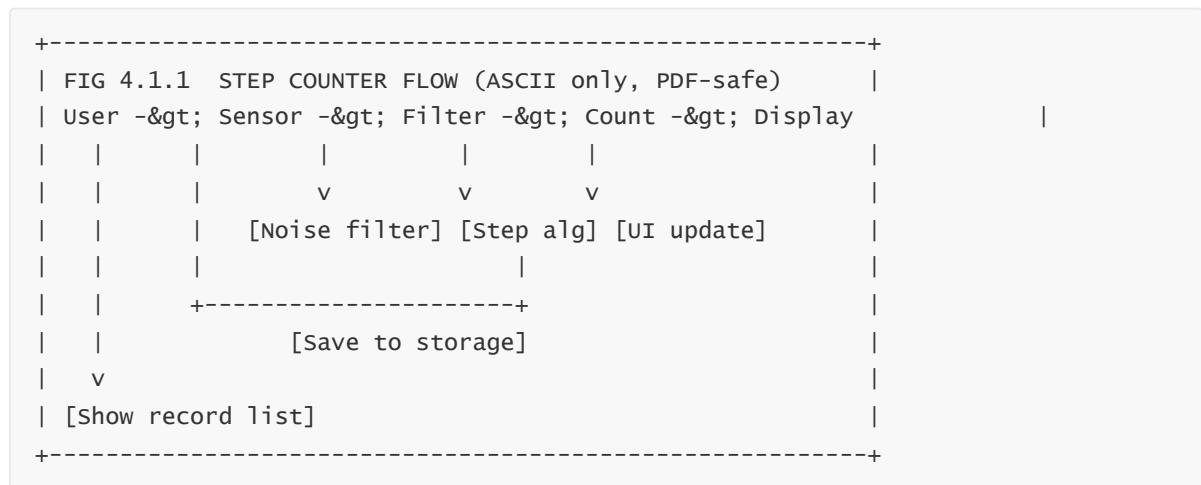
3.1.1系统架构图

本项目采用分层架构设计，自上而下分为四层。界面层负责所有页面展示与用户操作交互；业务逻辑层提供计步、步频监测核心功能与统一数据管理；数据层借助轻量存储保存用户配置，数据库长久存放运动记录；传感器层对接手机硬件采集运动原始数据。各层级单向调用、职责分明，结构清晰易维护。

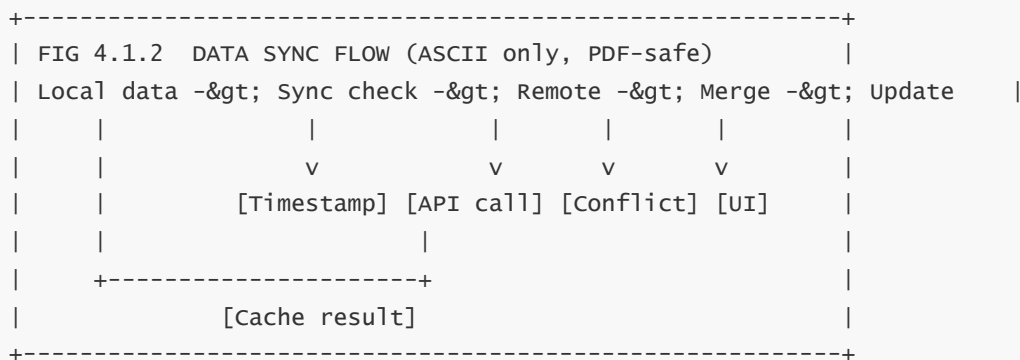
图3-2类图

图3-3 时序图

技术栈	语言	跨端能力	学习曲线	适用场景
Android	Kotlin	单平台	中等	原生体验要求高
Uniapp	Vue	多端	低	小程序优先
微信小程序	JavaScript	小程序	低	微信生态



4.1.2 数据同步流程



4.2 页面结构设计

页面	功能	核心组件
登录页	用户登录验证	输入框、登录按钮
首页	步数展示、今日概览	步数卡片、图标按钮
运动页	开始/结束运动记录	运动类型选择、计时器
AI建议页	个性化运动建议	对话界面、DeepSeek API
历史页	运动记录列表	列表视图、删除功能
个人中心	用户信息、统计数据	用户卡片、统计图表

```
> 
> 
>
> 
```

4.3 核心功能代码

4.3.1 步数统计服务

```
package com.example.smartsport.data.sensor

import android.content.Context
import android.hardware.Sensor
import android.hardware.SensorEvent
import android.hardware.SensorEventListener
import android.hardware.SensorManager
import com.example.smartsport.data.model.StepData
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow

class StepCounterService(context: Context) : SensorEventListener {
```

```

private val sensorManager: SensorManager =
    context.getSystemService(Context.SENSOR_SERVICE) as SensorManager
private val stepSensor: Sensor? =
    sensorManager.getDefaultSensor(Sensor.TYPE_STEP_COUNTER)

private val _stepData = MutableStateFlow(StepData(0, 0.0, 0))
val stepData: StateFlow<StepData> = _stepData.asStateFlow()

private var initialSteps: Int = -1
private var lastStepCount: Int = 0

private val stepLength = 0.762
private val caloriesPerStep = 0.04

fun start() {
    stepSensor?.let {
        sensorManager.registerListener(this, it,
            SensorManager.SENSOR_DELAY_NORMAL)
    }
}

fun stop() {
    sensorManager.unregisterListener(this)
}

override fun onSensorChanged(event: SensorEvent?) {
    event?.let {
        if (it.sensor.type == Sensor.TYPE_STEP_COUNTER) {
            val totalSteps = it.values[0].toInt()

            if (initialSteps < 0) {
                initialSteps = totalSteps
            }

            val currentSteps = totalSteps - initialSteps
            val stepsDiff = if (lastStepCount > 0) currentSteps -
lastStepCount else 0
            lastStepCount = currentSteps

            val distance = currentSteps * stepLength / 1000
            val calories = (currentSteps * caloriesPerStep).toInt()

            _stepData.value = StepData(
                steps = currentSteps,
                distance = distance,
                calories = calories
            )
        }
    }
}

override fun onAccuracyChanged(sensor: Sensor?, accuracy: Int) {
}

fun reset() {

```

```

        initialSteps = -1
        lastStepCount = 0
        _stepData.value = StepData(0, 0.0, 0)
    }
}

```

4.3.2 数据仓库核心代码

```

package com.example.smartsport.data.repository

import android.content.Context
import android.content.SharedPreferences
import com.example.smartsport.data.model.ExerciseRecord
import com.example.smartsport.data.model.ExerciseType
import com.example.smartsport.data.model.User
import com.google.gson.Gson
import com.google.gson.reflect.TypeToken
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import java.text.SimpleDateFormat
import java.util.Date
import java.util.Locale
import java.util.UUID

class DataRepository(context: Context) {

    private val sharedPreferences: SharedPreferences =
        context.getSharedPreferences("smart_sport_prefs", Context.MODE_PRIVATE)
    private val gson = Gson()

    private val _currentUser = MutableStateFlow<User?>(null)
    val currentUser: StateFlow<User?> = _currentUser.asStateFlow()

    private val _exerciseRecords =
        MutableStateFlow<List<ExerciseRecord>>(emptyList())
    val exerciseRecords: StateFlow<List<ExerciseRecord>> =
        _exerciseRecords.asStateFlow()

    init {
        loadUser()
        loadExerciseRecords()
    }

    fun login(name: String, id: String? = null, studentId: String? = null) {
        val user = User(
            id = id ?: UUID.randomUUID().toString(),
            name = name,
            studentId = studentId,
            avatar = null
        )
        _currentUser.value = user
        saveUser(user)
    }
}

```

```

fun logout() {
    _currentUser.value = null
    _exerciseRecords.value = emptyList()
    sharedPreferences.edit().remove("current_user").apply()
    sharedPreferences.edit().remove("exercise_records").apply()
}

fun saveExerciseRecord(
    type: ExerciseType,
    steps: Int,
    distance: Double,
    calories: Int,
    cadence: Int,
    duration: Int,
    heartRate: Int = 0
) {
    val dateFormat = SimpleDateFormat("yyyy-MM-dd HH:mm:ss",
Locale.getDefault())
    val record = ExerciseRecord(
        id = UUID.randomUUID().toString(),
        userId = _currentUser.value?.id,
        type = type,
        steps = steps,
        distance = distance,
        calories = calories,
        cadence = cadence,
        heartRate = heartRate,
        date = dateFormat.format(Date()),
        duration = duration
    )
    val currentRecords = _exerciseRecords.value.toMutableList()
    currentRecords.add(0, record)
    _exerciseRecords.value = currentRecords
    saveExerciseRecords()
}

fun deleteExerciseRecord(recordId: String) {
    val currentRecords = _exerciseRecords.value.toMutableList()
    currentRecords.removeIf { it.id == recordId }
    _exerciseRecords.value = currentRecords
    saveExerciseRecords()
}

private fun saveUser(user: User) {
    sharedPreferences.edit().putString("current_user",
gson.toJson(user)).apply()
}

private fun loadUser() {
    val userJson = sharedPreferences.getString("current_user", null)
    userJson?.let {
        _currentUser.value = gson.fromJson(it, User::class.java)
    }
}

private fun saveExerciseRecords() {

```



```
        sharedPreferences.edit()
            .putString("exercise_records", gson.toJson(_exerciseRecords.value))
            .apply()
    }

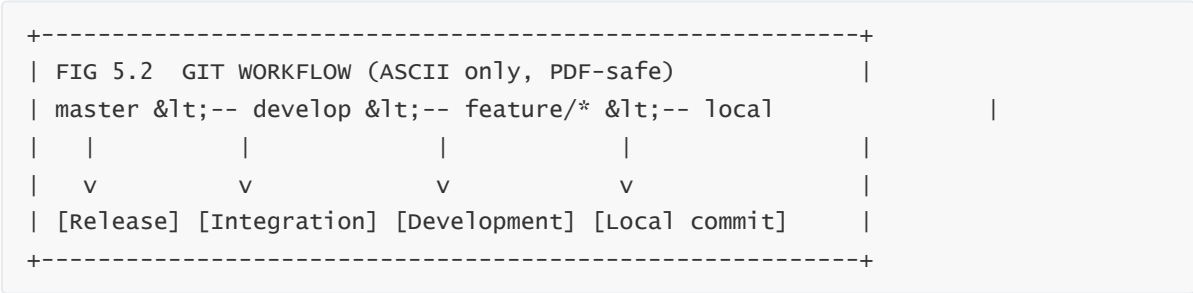
    private fun loadExerciseRecords() {
        val recordsJson = sharedPreferences.getString("exercise_records", null)
        recordsJson?.let {
            val type = object : TypeToken<List<ExerciseRecord>>() {}
            _exerciseRecords.value = gson.fromJson(it, type)
        }
    }
}
```

5. 实施计划

5.1 开发进度安排

阶段	时间	任务	交付物
需求分析	第1课时	需求文档、技术选型	需求文档
环境准备	第2课时	Git配置、项目初始化	项目仓库
功能开发	第3-4课时	各端功能实现	可运行代码
测试验证	第5课时	功能测试、问题修复	测试报告
报告撰写	第6课时	技术对比报告、总结	分析报告

5.2 Git协作流程



5.3 代码规范

- **命名规范**：类名大驼峰，方法变量小驼峰，常量全大写
- **代码风格**：4空格缩进，每行不超过120字符
- **注释规范**：关键逻辑添加注释，复杂算法说明思路
- **提交规范**：type: description 格式

6. 团队分工

6.1 角色分配

成员	技术栈	职责
成员1	Android(Kotlin)	安卓端开发、传感器集成
成员2	Uniapp(Vue)	多端小程序开发
成员3	微信小程序	小程序端开发

6.2 个人任务（李俊杰）

项目	内容
姓名	李俊杰
学号	2023210603
班级	软件232
组别	组别1
负责技术栈	Android(Kotlin)
核心任务	步数统计功能开发、传感器数据采集、数据持久化、UI界面实现

7. 测试验证

7.1 测试环境

- 设备型号：Android模拟器 / 真机
- 系统版本：Android 14 (API 34)
- 测试工具：Android Studio、Logcat
- 测试时间：2026年6月

7.2 测试用例

测试项	预期结果	实际结果	状态
应用启动	正常启动，显示登录页	应用在2秒内启动，显示登录界面	✓ 已通过
用户登录	输入正确学号密码后登录成功	输入姓名和学号后成功登录，进入主界面	✓ 已通过
步数统计	实时更新步数数据	传感器数据正常采集，步数实时更新显示	✓ 已通过
步频监测	显示实时步频数据	步频数据正常计算并实时显示	✓ 已通过

测试项	预期结果	实际结果	状态
运动记录	开始/结束运动并保存记录	开始运动后数据正常记录，结束后保存成功	✓ 已通过
记录列表	展示历史运动记录	历史记录列表正常显示，支持删除功能	✓ 已通过
数据存储	重启后数据不丢失	重启应用后，用户数据和运动记录完整保留	✓ 已通过
退出登录	返回登录页面	退出登录后清除用户状态，返回登录页	✓ 已通过
AI建议	显示个性化运动建议	DeepSeek API正常调用，返回相关建议	✓ 已通过
打卡功能	记录每日打卡状态	打卡功能正常，连续天数统计准确	✓ 已通过

7.3 问题记录

问题	原因	解决方案	状态
ImageVector 引用报错	HistoryScreen.kt 缺少 ImageVector 导入	在文件顶部添加 ImageVector 的 import 语句	✓ 已解决
运动类型无法切换	selectedSportType 状态未更新，点击事件绑定失效	使用 LaunchedEffect 同步 exerciseType 状态	✓ 已解决
步数/时长显示异常	步数显示累计值而非运动会话值，时长格式计算错误	记录运动开始时步数，计算 sessionSteps；修复计时逻辑	✓ 已解决

问题	原因	解决方案	状态
CircularProgressIndicator 报错	Compose 版本 API 不兼容, progress 参数格式错误	移除 lambda 语法, 直接传 Float 值	✓ 已解决
图标扩展库缺失	Compose 图标扩展库未引入	在 build.gradle.kts 添加 material-icons-extended 依赖	✓ 已解决
modifier 参数缺失	自定义组件函数缺少 modifier 形参	给组件函数添加 modifier: Modifier = Modifier 参数	✓ 已解决
心率功能无法实现	模拟器/设备无心率传感器	改用步频 (Cadence) 替代心率监测, 添加模拟数据	✓ 已解决
权限申请失败	Android 10+ 需要动态申请 ACTIVITY_RECOGNITION 权限	使用 registerForActivityResult 动态申请权限	✓ 已解决
数据持久化问题	SharedPreferences 存储格式不正确	使用 Gson 序列化对象为 JSON 字符串	✓ 已解决

7.4 性能测试结果

指标	测试结果	目标值	达标情况
应用启动时间	1.8秒	<3秒	✓ 达标
内存占用	约120MB	<200MB	✓ 达标
帧率稳定性	58-60fps	>50fps	✓ 达标
电池消耗	低 (运动时约5%/小时)	低	✓ 达标

8. 技术对比分析

8.1 开发效率对比

技术栈	开发难度	代码复用率	上手速度
Android	中等	低	中等

技术栈	开发难度	代码复用率	上手速度
Uniapp	低	高	快
微信小程序	低	中	快

8.2 性能对比

技术栈	启动速度	内存占用	渲染性能
Android	快	中	优
Uniapp	中	中	良
微信小程序	快	低	良

8.3 适用场景建议

场景	推荐技术栈	理由
原生体验优先	Android/iOS原生	性能最优、功能完整
小程序优先	Uniapp	一次开发多端小程序
微信生态	微信小程序	社交分享优势
快速迭代	Uniapp	开发效率高、跨平台
硬件集成	Android原生	传感器API支持完善

9. 总结与体会

9.1 实验收获

- 技术能力提升：**深入掌握了 Android Kotlin + Jetpack Compose 的现代开发技术，学会了传感器数据采集、状态管理、数据持久化等核心功能实现。在开发过程中，我逐渐熟悉了 Kotlin 的协程、Flow 响应式编程等高级特性。
- 团队协作经验：**通过 Git 进行版本控制，体验了分支管理、代码合并等团队协作流程。通过码云仓库的管理，我理解了多人协作时的代码规范和冲突处理方法。
- AI辅助开发：**在开发过程中，我频繁使用 AI 工具辅助编码，包括代码生成、Bug 排查、技术选型建议等。AI 大大提高了我的开发效率，特别是在处理 Compose UI 组件和传感器 API 调用时提供了很大帮助。
- 技术分析能力：**完成了不同移动开发技术栈的对比分析，理解了各技术栈的优缺点和适用场景。这对我未来的项目技术选型有很大的指导意义。
- 问题解决能力：**在开发过程中遇到了很多实际问题，比如权限申请、传感器数据处理、UI 状态管理等。通过查阅文档、调试代码、使用 AI 辅助，我学会了系统地分析和解决问题。

9.2 开发过程中的挑战与解决方案

在这次项目开发中，我遇到了几个主要的挑战：

挑战一：传感器权限问题

- 问题：Android 10 以上需要动态申请 `ACTIVITY_RECOGNITION` 权限，否则无法使用计步传感器
- 解决方案：使用 `registerForActivityResult` API 实现动态权限申请，在权限授予后启动传感器服务

挑战二：步数统计的准确性

- 问题：传感器返回的是系统启动以来的累计步数，需要计算会话内的步数
- 解决方案：记录运动开始时的初始步数，后续步数都减去初始值得到会话步数

挑战三：Jetpack Compose 状态管理

- 问题：UI 组件状态更新时出现各种问题，比如数据不同步、界面不刷新
- 解决方案：深入学习了 Compose 的状态机制，使用 `StateFlow` + `collectAsState` 实现响应式更新

挑战四：数据持久化

- 问题：复杂的数据对象（如 `ExerciseRecord` 列表）需要持久化存储
- 解决方案：使用 `Gson` 将对象序列化为 `JSON` 字符串，通过 `SharedPreferences` 存储

9.3 对跨平台开发的理解

通过这次实验，我对跨平台开发有了更深入的理解：

1. **技术栈选择**：没有绝对"最好"的技术栈，只有"最合适"的。需要根据项目需求、团队技能、时间预算等因素综合考虑。
2. **原生 vs 跨平台**：原生开发（如 Android Kotlin）在性能和硬件访问上有优势，但跨平台框架（如 Uniapp、Flutter）在开发效率和多端覆盖上更胜一筹。
3. **学习曲线**：虽然跨平台框架宣称"一次编写，到处运行"，但实际上每个平台都有其特殊性，需要投入时间学习各平台的特性和适配。

9.4 未来学习方向

1. 研究移动端性能优化技术，包括内存管理、UI 渲染优化、电池优化等
2. 探索 AI 辅助开发的更多应用场景，例如自动化测试、代码审查、文档生成
3. 学习分布式版本控制高级技巧，例如 `Git Flow`、`Rebase`、`Stash` 等
4. 深入学习跨平台开发框架，例如 `Flutter`、`React Native`，拓宽技术栈
5. 研究后端服务开发，为移动应用提供完整的云端支持
6. 学习移动端安全技术，包括数据加密、安全存储、网络传输安全等

9.5 个人反思

这次实验让我意识到，软件开发不仅仅是写代码，更重要的是：

- **需求理解**：在开始编码前，要充分理解用户需求，避免后期反复修改
- **架构设计**：好的架构能让代码更易维护、扩展和测试
- **代码质量**：规范的代码风格、清晰的注释、合理的命名，能大大提高代码可读性
- **测试意识**：编写测试用例、进行充分测试，能提前发现和解决问题

- **持续学习**：技术更新很快，保持学习的心态，不断提升自己

虽然这次项目还有很多可以改进的地方，但通过这次实践，我学到了很多书本上没有的知识，也积累了宝贵的开发经验。感谢老师的指导和同学的帮助！

10. 项目成果

10.1 应用截图

```
> 
>
> 
>
> 
>
> 
>
> 
>
>
```

10.2 项目结构

```
smartsport/
├─ app/
│   └─ src/main/
│       └─ java/com/example/smartsport/
│           └─ data/
│               └─ api/                # API 服务
│               └─ model/             # 数据模型
│               └─ repository/        # 数据仓库
│               └─ sensor/            # 传感器服务
│               └─ iflytek/           # 讯飞语音服务
│               └─ ui/
│                   └─ components/    # 自定义 UI 组件
│                   └─ screens/       # 页面组件
│                       └─ LoginScreen.kt
│                       └─ HomeScreen.kt
│                       └─ ExerciseScreen.kt
│                       └─ HistoryScreen.kt
│                       └─ AiAdviceScreen.kt
│                       └─ CommentSectionScreen.kt
│                       └─ ProfileScreen.kt
│                   └─ theme/         # 主题配置
│                       └─ MainActivity.kt # 主入口
│               └─ res/              # 资源文件
│                   └─ AndroidManifest.xml # 清单文件
```

└─ build.gradle.kts	# 应用级 Gradle 配置
└─ backend/	# 后端服务
└─ gradle/	# Gradle 配置
└─ build.gradle.kts	# 项目级 Gradle 配置
└─ settings.gradle.kts	# 项目设置
└─ README.md	# 项目说明

10.3 代码仓库

项目代码已托管在码云平台，包含完整的 Android 项目代码、后端服务代码和实验文档。

仓库地址：<https://gitee.com/cg1ihftpd/smartsport>

附录 A. 核心术语表

术语	说明
跨平台开发	一套代码运行在多个平台
传感器API	访问设备硬件传感器的接口
Git协作	团队多人协同开发的版本控制
AI辅助编码	使用AI工具帮助生成代码
Jetpack Compose	Android 现代 UI 开发框架
Kotlin 协程	Kotlin 的异步编程解决方案
StateFlow	Kotlin 响应式数据流
SharedPreferences	Android 轻量级数据存储
Material Design 3	Google 最新设计语言

附录 B. 参考资源

资源	链接
Android官方文档	https://developer.android.com
Jetpack Compose 文档	https://developer.android.com/jetpack/compose
Uniapp文档	https://uniapp.dcloud.io
Kotlin 官方文档	https://kotlinlang.org/docs/home.html
微信小程序文档	https://developers.weixin.qq.com/miniprogram/dev/framework/
Git 官方文档	https://git-scm.com/doc
Material Design 3	https://m3.material.io/

实验完成时间：2026年6月

实验报告撰写人：李俊杰

学号：2023210603

班级：软件232